

Anhanguera

Engenharia de Software

Roteiro de Aula Prática

Linguagem de Programação

Unidade 1 - Introdução à Linguagem Python

Aula 4 - Funções em Python

Experimento: Sistema de Gestão de Notas de Alunos

Yuri Gabriel Maciel de Macedo

Caxias do Sul - 2026

Folha de Rosto

Aluno:	Yuri Gabriel Maciel de Macedo
RA:	2026203499
Curso:	Engenharia de Software
Universidade:	Anhanguera - Caxias do Sul
Disciplina:	Linguagem de Programação
Unidade / Aula:	U1 - Introdução à Linguagem Python / A4 - Funções em Python
Professor(a):	Anderson
Data:	27/09/2026

1. Objetivo e Atividade Proposta

Esta atividade prática tem como objetivo compreender a importância das estruturas condicionais e aplicá-las em código Python. Foi desenvolvido um sistema simples de gestão de notas de alunos, que permite cadastrar notas em uma lista, calcular a média, determinar a situação do aluno (aprovado se a média for maior ou igual a 7; reprovado se for menor que 7) e exibir um relatório final. O código utiliza estruturas condicionais, de repetição e funções, e foi executado no Google Colab.

2. Codificação Completa

```
# Sistema de Gestão de Notas de Alunos

def cadastrar_notas():
    notas = [] # lista para guardar as notas
    quantidade = int(input("Quantas notas deseja cadastrar? "))

    for i in range(quantidade): # repete uma vez para cada nota
        nota = float(input("Digite a nota: "))
        notas.append(nota) # coloca a nota na lista
    return notas

def calcular_media(notas):
    return sum(notas) / len(notas) # soma tudo e divide pela quantidade

def determinar_situacao(media):
    if media >= 7: # média 7 ou mais aprova
        return "Aprovado"
    else:
        return "Reprovado"

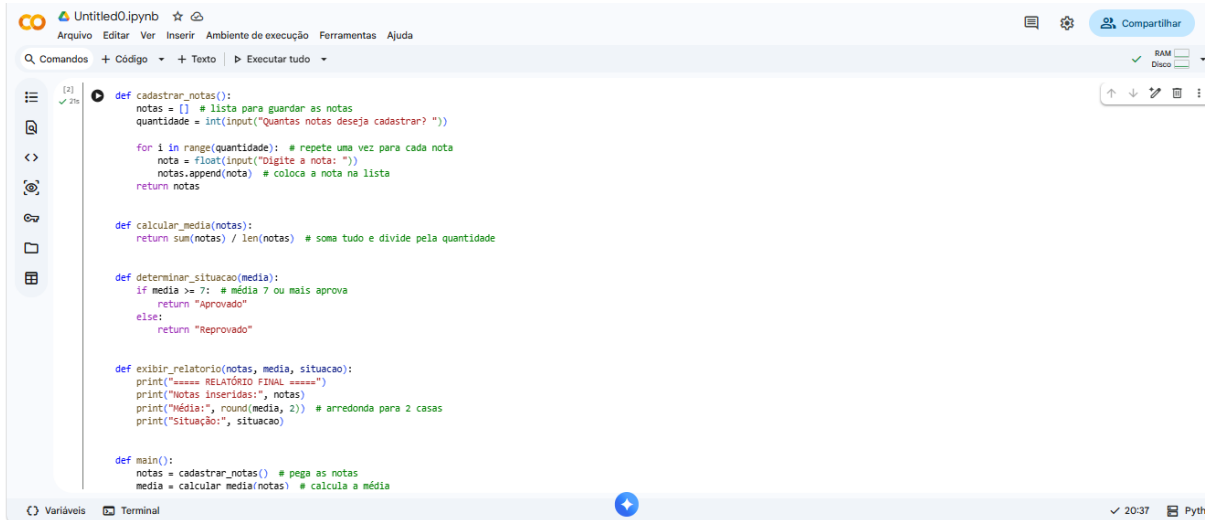
def exibir_relatorio(notas, media, situacao):
    print("==== RELATÓRIO FINAL =====")
    print("Notas inseridas:", notas)
    print("Média:", round(media, 2)) # arredonda para 2 casas
    print("Situação:", situacao)

def main():
    notas = cadastrar_notas() # pega as notas
    media = calcular_media(notas) # calcula a média
    situacao = determinar_situacao(media) # vê se aprovou
    exibir_relatorio(notas, media, situacao) # mostra o resultado

main() # inicia o programa
```

3. Prints de Execução

Figura 1 - Código no Google Colab (parte inicial)



```
[2] ✓ 27s
def cadastrar_notas():
    notas = [] # lista para guardar as notas
    quantidade = int(input("Quantas notas deseja cadastrar? "))

    for i in range(quantidade): # repete uma vez para cada nota
        nota = float(input("Digite a nota: "))
        notas.append(nota) # coloca a nota na lista
    return notas

def calcular_media(notas):
    return sum(notas) / len(notas) # soma tudo e divide pela quantidade

def determinar_situacao(media):
    if media >= 7: # média 7 ou mais aprova
        return "Aprovado"
    else:
        return "Reprovado"

def exibir_relatorio(notas, media, situacao):
    print("==== RELATÓRIO FINAL =====")
    print("Notas inseridas:", notas)
    print("Média:", round(media, 2)) # arredonda para 2 casas
    print("Situação:", situacao)

def main():
    notas = cadastrar_notas() # pega as notas
    media = calcular_media(notas) # calcula a média
```

Figura 2 - Final do código e execução com o relatório final



```
[2] ✓ 27s
if media >= 7: # média 7 ou mais aprova
    return "Aprovado"
else:
    return "Reprovado"

def exibir_relatorio(notas, media, situacao):
    print("==== RELATÓRIO FINAL =====")
    print("Notas inseridas:", notas)
    print("Média:", round(media, 2)) # arredonda para 2 casas
    print("Situação:", situacao)

def main():
    notas = cadastrar_notas() # pega as notas
    media = calcular_media(notas) # calcula a média
    situacao = determinar_situacao(media) # vê se aprovou
    exibir_relatorio(notas, media, situacao) # mostra o resultado

main() # inicia o programa

Quantas notas deseja cadastrar? 3
Digite a nota: 8
Digite a nota: 6.5
Digite a nota: 7.5
==== RELATÓRIO FINAL =====
Notas inseridas: [8.0, 6.5, 7.5]
Média: 7.33
Situação: Aprovado
```

4. Avaliando os Resultados

Questão: Você está criando um sistema para cadastrar notas, calcular média e mostrar se o aluno foi aprovado ou reprovado. Como o uso de funções e listas pode facilitar futuras melhorias nesse sistema?

O uso de funções e listas facilita futuras melhorias porque deixa o sistema organizado e modular. Cada função tem uma única responsabilidade (cadastrar, calcular, classificar e exibir), então é possível alterar uma parte sem quebrar as outras. Por exemplo, se a média mínima de aprovação mudar de 7 para 6, basta editar a função `determinar_situacao`. Além disso, as funções podem ser reutilizadas em outros pontos do código, o que evita repetição e reduz erros.

As listas permitem armazenar um número variável de notas sem criar uma variável para cada uma. Isso facilita evoluir o sistema para calcular maior e menor nota, tirar médias ponderadas, ordenar notas ou até guardar várias listas, uma por aluno. O código também fica mais fácil de testar e de entender, e mais simples de ampliar, por exemplo com cadastro de vários alunos, gravação em arquivo ou uma interface gráfica.

5. Referências

GOOGLE. **Google Colaboratory**. Disponível em: <https://colab.research.google.com/>. Acesso em: 27 set. 2026.